Experimental Report SYSC4001

Assignment 1

By: Isabella Hernandez

Student Number: #101352857

Part A. Scheduling Tradeoffs [15 Marks]

Workload:

P1 arrives at t = 0. P2, P3, P4 (bursts 3, 2, 4) arrive at t = 2, 4, 6. Context-switch cost of 1.

Commands:

./scheduler -i report_scenarios/scheduling.csv -a fcfs -q 1 -c 1 -o out_fcfs

./scheduler -i report_scenarios/scheduling.csv -a srtf -q 1 -c 1 -o out_srtf

./scheduler -i report_scenarios/scheduling.csv -a rr -q 1 -c 1 -o out_rr_q1

./scheduler -i report_scenarios/scheduling.csv -a rr -q 10 -c 1 -o out_rr_q10

Results:

Run	Command	Makespan	Avgerage response	Avergage turnaround	P1(long) turnaround
1	-a fcfs -q 1 -c 1	23	9.00	13.75	11
2	-a srtf -q 1 -c 1	24	2.25	10.25	24
3	-a rr -q 1 -c 1	38	4.00	23.50	38
4	-a rr -q 10 -c 1	23	9.00	13.75	11

1. How does changing the RR quantum affect response time and time spent context switching?

Changing the quantum from 10 to 1 improves average response from 9.00 to 4.00, because the short processes get to the CPU sooner. The cost is context switching, total switch time went from 4 to 19 ticks, since every tick of CPU work is now preceded by a switch (RR q=1 timeline: 4-5 CONTEXT_SWITCH, 5-6 CPU P2, 6-7 CONTEXT_SWITCH, 7-8 CPU P1). This moved up the makespan from 23 to 38 and dropped utilization from 83% to 50%. Average turnaround also got worse, rising from 13.75 to 23.50.

2. How does SRTF compare with FCFS in average turnaround time? What happens to the long process?

SRTF shortens average turnaround from 13.75 to 10.25, because the short processes jump ahead of P1. P1 then pays for it because its turnaround rose from 11 to 24. It was preempted and finished last. At t = 2, P2 arrives which it then needs less time than P1, and the SRTF timeline shows 1-2 CPU P1, 2-3 CONTEXT_SWITCH, 3-6 CPU P2, with P1 only continuing at 15-24. The makespan grew by 1 because the preemption cost one extra context switch.

Part B. CPU and I/O Overlap [15 Marks]

Workload:

P1 (two 4-tick bursts, 5-tick I/O between, arrives t=0). P2 (one 4-tick burst, arrives t=4). baseline.csv = overlap.csv without P2.

Commands:

./scheduler -i report_scenarios/overlap.csv -a fcfs -q 1 -c 1 -o out_overlap

./scheduler -i report_scenarios/baseline.csv -a fcfs -q 1 -c 1 -o out_baseline

Results:

File	CPU busy time	Idle time	Makespan	CPU utilization
overlap.csv	12	0	15	80%
baseline.csv	8	5	15	53%

1. When does the second process use CPU time that would otherwise be idle? Give one specific interval from timeline.csv.

P2 runs from t = 6 to t = 10 (timeline row 6, 10, CPU, 2), while P1 is waiting on I/O from t = 5 to t =10. In baseline.csv that window is idle, which is why its idle time is 5 and overlap.csv's is 0.

2. Why can CPU utilization improve without reducing the makespan?

The makespan is fixed by P1 alone: a context switch, a 4-tick burst, 5 ticks of I/O, another switch, and a second 4-tick burst add up to 15. P2 runs entirely inside the I/O wait, so it finishes without delaying P1. Utilization rises because the same 15 ticks now hold 12 busy ticks instead of 8, and the 5 idle ticks drop to 0.